

REPORT DI PROGETTO

TRACCIA 1: Sistema di gestione degli interventi di manutenzione in un condominio

(Giosuè Senatore, Jacopo Salimbene, Manuel Vitolo)

1. Motivazione scelta dell'ADT (Abstract Data Type)

1. Gestione delle Richieste Attive (Max-Heap / Coda di Priorità)

- **Come funziona:** In un condominio, le emergenze assolute (es. tubo rotto) devono avere la precedenza su guasti minori (es. lampadina fulminata), indipendentemente dall'ordine di arrivo.
- **La scelta tecnica:** Abbiamo implementato una **Coda di Priorità basata su un array strutturato come Max-Heap**. Questa soluzione garantisce che la richiesta con l'urgenza massima si trovi sempre alla radice (indice 0) e sia estraibile in tempo $O(1)$. L'inserimento di nuove richieste si auto-bilancia in tempo logaritmico $O(\log N)$, risultando estremamente più efficiente di una lista ordinata.

2. Gestione dei Tecnici (Tabella Hash)

- **Come funziona:** Il sistema deve poter rintracciare istantaneamente il profilo di un tecnico partendo dal suo codice identificativo (ID), senza dover scorrere l'intera anagrafica.
- **La scelta tecnica:** Abbiamo sostituito la lista lineare con una **Tabella Hash**. Utilizzando una funzione di hash basata sull'ID e gestendo le collisioni tramite *chaining* (liste concatenate nei bucket), l'accesso, la ricerca e l'assegnazione di un tecnico avvengono in tempo costante $O(1)$ nel caso medio.

3. Pianificazione e Agende (Lista Concatenata Semplice)

- **Come funziona:** Quando si assegna un lavoro, il sistema deve verificare che il tecnico non sia impegnato in un altro appartamento nella stessa fascia oraria.
- **La scelta tecnica:** Poiché l'agenda di un singolo tecnico contiene generalmente un numero limitato di appuntamenti attivi, abbiamo implementato una **Lista Concatenata Semplice**. L'inserimento di un nuovo appuntamento in testa all'agenda avviene in $O(1)$, e lo scorrimento per il controllo dei conflitti di orario risulta sufficientemente rapido e leggero in termini di memoria.

4. Storico degli Interventi (Albero Binario di Ricerca - BST)

- **Come funziona:** Una volta riparato il guasto, la pratica passa in archivio. Questo archivio è destinato a crescere a dismisura negli anni. L'amministratore deve poter ritrovare vecchie pratiche velocemente tramite l'ID.

- **La scelta tecnica:** L'archivio è stato implementato come un **Albero Binario di Ricerca (BST)**. Questo ci permette di dimezzare lo spazio di ricerca ad ogni passo, garantendo ricerche rapidissime in $O(\log N)$. Inoltre, la struttura ad albero permette di generare report ordinati per ID utilizzando una semplice visita *In-Order*.
-

2. Progettazione: Architettura del Sistema

Il sistema segue un'architettura modulare (Multi-file) divisa nei seguenti moduli funzionali:

- **Modulo Richieste (*richieste.h / .c*):** Gestisce l'inserimento nel Max-Heap, l'estrazione delle emergenze e le funzioni di utilità (heapify).
 - **Modulo Tecnici (*tecnici.h / .c*):** Gestisce l'anagrafica e l'indicizzazione tramite Tabella Hash.
 - **Modulo Storico (*storico.h / .c*):** Modulo dedicato alla conservazione a lungo termine e alla ricerca rapida degli interventi passati tramite Albero Binario (BST).
 - **Modulo Pianificazione (*pianificazione.h / .c*):** Rappresenta il "motore" del sistema. Preleva i dati dall'Heap e dalla Tabella Hash, verificando la compatibilità delle specializzazioni e prevenendo i conflitti temporali nelle agende.
 - **Modulo Reportistica (*report.h / .c*):** Analizza le strutture dati per calcolare le metriche di sistema.
 - **Modulo Main (*main.c*):** Gestisce l'interfaccia utente (CLI a menu), il ciclo di vita dell'applicazione e la fondamentale liberazione della memoria (Garbage Collection manuale).
-

3. Specifica Sintattica e Semantica

Di seguito le specifiche delle operazioni fondamentali (Core API) aggiornate al nuovo sistema.

3.1 Operazione: *inserisciRichiestaHeap*

- **Sintassi:** *void inserisciRichiestaHeap(SistemaCondominio* sys, Richiesta* nuova)*
- **Pre-condizioni:** L'oggetto *sys* deve essere inizializzato e l'Heap non deve aver superato la capacità massima (*MAX_HEAP_CAPACITY*).
- **Post-condizioni:** La nuova richiesta viene inserita nell'array dell'Heap. L'algoritmo di *heapifyUp* ripristina la proprietà del Max-Heap posizionando la richiesta in base all'urgenza.
- **Effetti Collaterali:** *heap_size* aumenta di 1.

3.2 Operazione: **pianificaIntervento**

- **Sintassi:** `bool pianificaIntervento(SistemaCondominio* sys, int id_richiesta, int id_tecnico, Data d, FasciaOraria f)`
- **Pre-condizioni:** La richiesta e il tecnico devono esistere (ricercati rispettivamente in $O(N)$ nell'Heap e in $O(1)$ nell'Hash Table). Il tecnico deve essere disponibile e avere la specializzazione adatta.
- **Post-condizioni:** Se la ricerca nell'agenda del tecnico non rileva sovrapposizioni, la richiesta passa in stato **PIANIFICATA** e viene inserita nell'agenda. Ritorna **true**. Se c'è conflitto o incompatibilità, ritorna **false**.
- **Effetti Collaterali:** La lista concatenata dell'agenda del tecnico acquisisce un nuovo nodo in testa.

3.3 Operazione: **Avanzamento di Stato e Archiviazione (Gestito via CLI/Motore)**

- **Logica Applicata:** Quando l'amministratore modifica lo stato di una richiesta in **CONCLUSA** o **ANNULLATA**.
- **Pre-condizioni:** La richiesta deve essere presente nel Max-Heap.
- **Post-condizioni:** La richiesta viene estratta e rimossa dall'array del Max-Heap tramite la funzione `rimuoviRichiestaDaHeap` e il suo puntatore viene inserito nel BST dello storico tramite `archiviaNelloStorico`.
- **Effetti Collaterali:** Diminuzione della dimensione dell'Heap e creazione di un nuovo nodo nell'Albero Binario.

Caso di Test	Obiettivo / Razionale	Esito Atteso
Registrazione Richieste (Max-Heap)	Verificare che inserendo una richiesta "Critica" dopo una "Bassa", l'algoritmo <i>heapify</i> posizioni la critica in cima all'indice 0.	La stampa delle richieste mostra l'emergenza come prima operazione disponibile.
Assegnazione senza Specializzazione	Garantire che un "Elettricista" non venga assegnato a un problema "Idraulico".	Assegnazione fallita, avviso a schermo e blocco operazione.

Pianificazione Sovrapposta (Conflitti)	L'agenda non deve permettere a un tecnico di essere in due posti alla stessa ora. Il controllo sulla lista dell'agenda deve rilevare la sovrapposizione matematica.	Pianificazione fallita, richiesta di nuova fascia oraria.
Archiviazione Storica (BST)	Testare la rimozione dall'Heap e il corretto inserimento nell'Albero Binario quando una richiesta viene conclusa.	La pratica scompare dalle "Attive" ed è ricercabile in $O(\log N)$ dallo Storico.
Ricerca tramite Tabella Hash	Fornire un ID tecnico inesistente o che genera collisione (stesso bucket).	Restituzione corretta del tecnico o di valore NULL gestito senza crash di memoria.